

1.Scenario 02

📁 File Upload

"Generate test cases for file upload."

What's missing

No file types, no size limits, no success/failure criteria, no security angle (malicious file upload).

chatgpt.com/c/6aad017b-5124-83e8-8cd6-69224bd7b445

Generate test cases for file upload.

Below are functional file-upload test cases covering positive, negative, validation, and boundary scenarios.

TC ID	Priority	Test Scenario	Precondition	Steps	Expected Result	Test Data
TC-FU-001	P0	Upload a valid file	User is on upload page	1. Click Upload 2. Select a valid file 3. Submit	File uploads successfully and confirmation is displayed	document.pdf
TC-FU-002	P0	Upload a file with an allowed extension	Upload feature is available	Select and upload an allowed file type	File is accepted and uploaded	report.pdf, image.jpg
TC-FU-003	P1	Upload a file with an unsupported extension	Upload feature is available	Select unsupported file and submit	Upload is rejected with appropriate validation message	script.exe
TC-FU-004	P1	Upload a file exceeding maximum size	Maximum file size is configured	Select a file larger than the limit	File is rejected with a clear size-limit message	largefile.zip
TC-FU-005	P1	Upload a file at the maximum allowed size	Maximum size is configured	Select a file exactly at the size limit	File uploads successfully	File = exactly configured limit
TC-FU-006	P1	Upload an empty/zero-byte file	Upload feature is available	Select a 0-byte file	File is rejected with appropriate error message	empty.txt
TC-FU-007	P1	Upload without selecting a file	Upload page is open	Click Submit/Upload without choosing a file	Validation message is displayed: upload does not proceed	No file
TC-FU-008	P1	Upload multiple files when multiple upload is supported	Multiple-file upload is enabled	Select multiple valid files and submit	All valid files are uploaded successfully	file1.pdf, file2.pdf
TC-FU-009	P2	Upload duplicate	A file has already been uploaded	Upload the same file again	System follows defined duplicate-file behavior: reject, replace, or create another copy	document.pdf
TC-FU-010	P2	Upload file with special characters in filename	Upload feature is available	Select a file containing special characters	File is handled correctly, according to filename rules	
TC-FU-011	P2	Upload file with a very long filename	Upload feature is available	Select file with filename exceeding configured limit	File is rejected or filename is handled according to requirements	255+ character filename
TC-FU-012	P2	Cancel file selection	Upload page is open	Click Upload → select a file → cancel selection	No file is attached and page remains usable	N/A
TC-FU-013	P1	Upload file with uppercase extension	Upload supports the file type	Select file with uppercase extension	File is accepted if extension matching is case-insensitive	REPORT.PDF
TC-FU-014	P1	Verify upload progress	Upload feature is available	Upload a relatively large valid file	Progress indicator is displayed and reaches completion	large_valid.pdf
TC-FU-015	P1	Verify failed upload handling	Upload service is unavailable/interrupted	Start uploading a valid file	Appropriate failure message is displayed and system does not show false success	Valid PDF + network interruption
TC-FU-016	P2	Upload the same file from different supported locations	Upload feature is available	Select valid file from supported location and upload	File uploads successfully regardless of valid source location	Valid PDF
TC-FU-017	P1	Verify file type validation when MIME type differs from extension	Upload validation is enabled	Attempt to upload a file whose actual content doesn't match its extension	File is rejected if content/MIME validation is required	Renamed executable → .pdf
TC-FU-018	P2	Verify uploaded file can be opened/downloaded	A valid file has been uploaded	Upload file → open/download uploaded file	Retrieved file is accessible and content is intact	sample.pdf

Key P0 scenarios

Key P0 scenarios

For a critical file-upload feature, I would treat these as the minimum smoke coverage:

- TC-FU-001: Valid file upload
- TC-FU-002: Allowed file type
- TC-FU-003: Unsupported file type
- TC-FU-004: Maximum file-size validation
- TC-FU-007: Upload without selecting a file

If you want, I can also create **exactly 10 file-upload test cases in your earlier format** (TC ID | Priority | Description | Precondition | Steps | Expected/Actual Result | Test Data) with a ↪ P0/P1/P2 mix and ↪ Team Lead review comments.

With ICEPOT

Instructions:

- *Generate Testcases for Testing File Uploading Funtionality*
- *[STRICT] Functional only — positive and negative*
- *[MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data*
- *[CRITICAL] Mix of P0, P1, P2*
- *[MANDATORY] Exactly 10 test cases*
- *Then switch persona and review, adding comments*

Context:

File Upload for uploading signature in declaration page of the application

Example:

TC-ID: TC001 Priority: P1

Description: Upload a valid Signature Image file

Precondition: a valid file which is less than 250KB

Steps: 1) Login to the application 2) Go the the declarations page 3)Upload the Signature file 3) Verify the preview 4) click on the submit Button

Expected Result: The submit button got submitted successfully with valid message "Image Uploaded successfully". Image is present in the backend cloud bucket location.

*Test Data: a file of 150KB with 10*10*

Persona:

- *Generate: QA Engineer, expert in functional testing*
- *Review: Team Lead, reviewing the test cases*

Output Format:

10 test cases in a table, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.

File Upload — Declaration Signature Functionality						
TC ID	Priority	Description	Precondition	Steps	Expected/Actual Result	Test Data
TC001	P0	Upload a valid signature image and submit the declaration	User is logged in; declaration page is accessible; valid signature image is available and within the 250 KB limit	1. Navigate to Declaration page. 2. Click Upload Signature . 3. Select the valid signature image. 4. Verify the preview. 5. Click Submit .	Expected: Signature is uploaded successfully; preview is displayed correctly; declaration is submitted successfully, and a confirmation message such as "Image Uploaded successfully" is displayed.	Valid signature image.JPG, 150 KB, 10×10 pixels
TC002	P0	Upload a signature image exceeding the maximum file size	User is on the Declaration page; maximum allowed size is 250 KB	1. Click Upload Signature . 2. Select an image larger than 250 KB. 3. Observe the validation message.	Expected: File upload is rejected and an appropriate file-size validation message is displayed. Declaration cannot be submitted with the invalid file.	JPG signature image, 300 KB
TC003	P0	Upload a signature with an unsupported file format	User is on the Declaration page	1. Click Upload Signature . 2. Select a file with an unsupported extension. 3. Observe the result.	Expected: File is rejected and an appropriate unsupported-file-type validation message is displayed.	signature.pdf
TC004	P1	Submit the declaration without uploading a signature	User is logged in and on the Declaration page; signature is mandatory	1. Navigate to Declaration page. 2. Do not upload a signature. 3. Click Submit .	Expected: Declaration submission is prevented and a validation message indicates that the signature is required.	No signature file
TC005	P1	Upload a valid PNG signature image	User is on the Declaration page; PNG is a supported format	1. Click Upload Signature . 2. Select a valid PNG signature image within the size limit. 3. Verify the preview. 4. Submit the declaration.	Expected: PNG file is accepted; preview is displayed correctly, and declaration is submitted successfully.	PNG, 100 KB, valid dimensions
TC006	P1	Upload a zero-byte signature image	User is on the Declaration page	1. Click Upload Signature . 2. Select a zero-byte image file. 3. Observe the validation result.	Expected: Zero-byte file is rejected and an appropriate validation message is displayed.	signature.jpg, 0 KB
TC006	P1	Upload a zero-byte signature image	User is on the Declaration page	1. Click Upload Signature . 2. Select a zero-byte image file. 3. Observe the validation result.	Expected: Zero-byte file is rejected and an appropriate validation message is displayed.	
TC007	P1	Replace an already uploaded signature with another valid signature	User has already uploaded a valid signature on the Declaration page	1. Upload Signature A. 2. Verify its preview. 3. Select Replace/Upload Signature . 4. Upload Signature B. 5. Verify the preview. 6. Submit.	Expected: Previous signature is replaced by the new valid signature. The latest signature is displayed and submitted.	Signature A: JPG, 120 KB; Signature B: PNG, 100 KB
TC008	P2	Upload a valid signature at exactly the maximum file size	User is on the Declaration page; maximum allowed size is 250 KB	1. Click Upload Signature . 2. Select a valid image exactly 250 KB in size. 3. Verify the preview. 4. Submit the declaration.	Expected: File is accepted because it is exactly at the configured maximum size. Preview and submission should complete successfully.	JPG, exactly 250 KB
TC009	P2	Upload a valid signature with an invalid/unsupported image dimension	User is on the Declaration page; application has defined signature dimension requirements	1. Click Upload Signature . 2. Select an image that exceeds the supported dimensions. 3. Observe the validation result.	Expected: File is rejected if the dimensions are outside the configured limits, with an appropriate validation message.	JPG, 5000×5000 pixels, <250 KB
TC010	P2	Upload a valid signature with a filename containing special characters	User is on the Declaration page; file is otherwise valid	1. Click Upload Signature . 2. Select a valid signature image with special characters in its filename. 3. Verify the preview. 4. Submit the declaration.	Expected: File is accepted if special characters are permitted by the application; preview is displayed correctly, and declaration submission succeeds.	file:Signature-91.jpg, 150 KB, valid dimensions

Team Lead Review Summary

Team Lead Review Summary

The test suite provides **10 functional positive and negative scenarios** with coverage across the primary upload workflow, mandatory-field validation, supported/unsupported formats, file-size boundaries, zero-byte files, replacement behavior, and image-dimension validation. Priorities are distributed across **P0, P1, and P2** based on business impact.

Team Lead Comments

1. **TC001 should remain P0** because successful signature upload and declaration submission represent the core business flow.
2. **TC002 and TC003 are appropriately prioritized as P0** because incorrect file validation can directly affect the integrity of the declaration process.
3. **TC004 should remain P1** since mandatory signature validation is an important functional control but is a negative path.
4. **TC007 should explicitly verify that the old signature is no longer used after replacement.**
5. **TC008 is a necessary boundary-value test** and should remain P2 unless the 250 KB limit is a critical business rule.
6. **TC009 depends on documented image-dimension requirements.** The exact minimum and maximum dimensions should be added to the requirement/test data before execution.
7. **Actual Result should be populated during execution** rather than being predetermined in the test case. The current "Expected/Actual Result" column should therefore be split into separate **Expected Result** and **Actual Result** columns in the execution version.
8. Ensure the final test execution verifies both the **UI upload status** and, where applicable, successful persistence of the signature in the application's configured backend storage.



Observations

1. Context based testcases
2. Detailed review by persona mentioned
3. Testcases follows the format mentioned in Examples

2. Scenario 05

🌐 API Testing

"Write API test cases for user registration."

What's missing

no request/response structure, no status codes to validate, no tool context (Postman/RestAssured).

Write API test cases for user registration.

Below are 10 functional API test cases for a typical User Registration API, covering positive and negative scenarios.

Assumed endpoint: `POST /api/v1/register`
Content-Type: `application/json`

TC ID	Priority	Description	Precondition	Steps	Expected / Actual Result	Test Data
REG_API_001	P0	Register a user with valid mandatory details	Registration API is available	1. Send POST request with valid name, email, password and phone. 2. Verify response.	Expected: User is created successfully. HTTP 201 Created returned with registration/user ID. Actual: —	name=John Doe, email=john@test.com, password=Test@1234, phone=9876543210
REG_API_002	P0	Register with an email that already exists	User with the email already exists	1. Send registration request using existing email. 2. Verify response.	Expected: Registration rejected with appropriate error. HTTP 409 Conflict or documented equivalent. Actual: —	Existing email: john@test.com
REG_API_003	P1	Register without mandatory email field	API is available	1. Remove email from request body. 2. Send POST request.	Expected: Request rejected with validation error identifying email as required. HTTP 400 Bad Request. Actual: —	Email omitted
REG_API_004	P1	Register with invalid email format	API is available	1. Provide malformed email. 2. Send POST request.	Expected: Request rejected with email validation error. HTTP 400 Bad Request. Actual: —	email=john@
REG_API_005	P1	Register with password that does not meet password policy	Password policy is configured	1. Provide a password shorter than the minimum	Expected: Registration rejected with password-policy validation error. Actual: —	password=123
REG_API_006	P1	Register without password	API is available	1. Remove password from request. 2. Send POST request.	Expected: Request rejected with "password is required" or equivalent validation error. HTTP 400. Actual: —	
REG_API_007	P2	Register with invalid phone number	API is available	1. Enter an invalid/non-supported phone number. 2. Send request.	Expected: Request rejected with phone-number validation error. Actual: —	phone=1234567
REG_API_008	P2	Register with leading/trailing spaces in input fields	API is available	1. Add spaces around name/email. 2. Send registration request.	Expected: API either trims whitespace and registers successfully or rejects it according to the documented validation rules. Actual: —	name=" John Doe ", email=" john2@test.com "
REG_API_009	P2	Register with unsupported HTTP method	API endpoint is available	1. Send GET request to registration endpoint. 2. Verify response.	Expected: Request rejected with HTTP 405 Method Not Allowed or documented equivalent. Actual: —	GET /api/v1/register
REG_API_010	P2	Register with malformed JSON request body	API is available	1. Send malformed JSON. 2. Verify response.	Expected: Request rejected with HTTP 400 Bad Request; no user is created. Actual: —	{ "name": "John", "email": "john@test.com"

Key API validations to verify

- HTTP status code
- Response body and error message/schema
- User ID/registration ID generation
- Duplicate-user prevention
- Mandatory-field validation
- Email/password/phone validation
- No user creation when validation fails
- Correct Content-Type
- Response time within the defined SLA
- Sensitive data such as password should not be returned in the API response

With ICEPOT

Instructions:

- *Generate API Testcases for user registration*
- *[STRICT] Functional only — positive and negative and security*
- *[MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data*
- *[CRITICAL] Mix of P0, P1, P2*
- *[MANDATORY] Exactly 10 test cases*
- *Then switch persona and review, adding comments*

Context:

Call the post API to register the user to the application

*postman request POST 'https://abc.com/api/v1/register' *

```
--body '{  
  "user_fname": "Clinda",  
  "user_lname": "Thomas",  
  "dob": "1990/05/05",  
  "email": "abcjhj@gmail.com",  
  "phone": "2556544578",  
  "password": "hvjhgshj"  
}'
```

Example:

TC-ID: TC001 Priority: P1

Description: validate the successful user creation via POST request in POST man

Precondition: User information with firstname, lastname, dob, email, phone and password

Dob in yyyy/mm/dd format, phone : must me 10 digits, password :atleast 12 char length,minimum 1 capital letter, 1 small letter, 1 special char

Steps: 1)Call the post API with valid request body.

Expected Result: response status 201. response {"user registerd successfully"}

Test Data: json formatted

```
{  
  "user_fname": "Clinda",  
  "user_lname": "Thomas",  
  "dob": "1990/05/05",  
  "email": "abcjhj@gmail.com",  
  "phone": "2556544578",
```

```
"password": "hvjhgshj"
}
```

Persona:

- Generate: QA Engineer, expert in functional testing
- Review: Team Lead, reviewing the test cases

Output Format:

10 test cases in a Google sheet, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in Test cases, review comments and summary.

Created the Google Sheets-compatible Excel file with exactly 10 API test cases, including P0/P1/P2 priorities, positive/negative/security scenarios, and a separate Team Lead Review sheet.

[Download the User Registration API Test Cases](#)

The Team Lead review also identifies that the password in your original sample (hvjhgsahj) does **not** meet the stated 12-character/complexity requirement, so compliant test data was used where appropriate. [↗](#)

User_Registration_API_Test_Cases.xlsx

Spreadsheet

Library / User_Registration_API_Test_Cases.xlsx

100%

	A	B	C	D	E	
1	TC ID	Priority	Description	Precondition	Steps	Exp
2	TC001	P0	Validate successful user registration with all valid mandatory details via POST API.	Registration API is available; email and phone are not already registered. DOB must be yyyy/mm/dd; phone exactly 10 digits; password at least 12 characters with 1 uppercase, 1 lowercase and 1 special character.	1. Open Postman. 2. Select POST and use https://abc.com/api/v1/register. 3. Set Content-Type to application/json. 4. Send valid JSON. 5. Verify status and response body.	Exp cre Act
3	TC002	P0	Validate that registration is rejected when an already registered email is submitted.	A user already exists with the supplied email.	1. Send POST request with otherwise valid data. 2. Use an already registered email. 3. Verify status and response. 4. Verify no duplicate user is created.	Exp eqi Act
4	TC003	P1	Validate registration rejection when the password is less than 12 characters or fails complexity requirements.	Registration API is available and email/phone are unique.	1. Send POST request with valid fields. 2. Provide a password shorter than 12 characters. 3. Verify response.	Exp is r Act
5	TC004	P1	Validate registration rejection for an invalid DOB format.	Registration API is available; all other values are valid.	1. Send POST request. 2. Enter DOB in a format other than yyyy/mm/dd. 3. Verify response.	Exp not Act
6	TC005	P1	Validate registration rejection when the phone number is not exactly 10 digits.	Registration API is available; all other values are valid.	1. Send POST request. 2. Provide a phone number with fewer or more than 10 digits. 3. Verify response.	Exp not Act
7	TC006	P1	Validate registration rejection when a mandatory field is missing.	Registration API is available.	1. Prepare a valid payload. 2. Remove user_fname. 3. Send POST request. 4. Verify response.	Exp fiel Act
8	TC007	P1	Validate safe handling of SQL-injection input in registration fields.	Registration API is available and security validation is enabled.	1. Send POST with SQL-injection text in user_fname and a unique email. 2. Verify status/body. 3. Verify no unintended database operation or unauthorized account creation occurs.	Exp dat Act
9	TC008	P2	Validate safe handling of script/HTML payloads in registration fields.	Registration API is available and input validation is enabled.	1. Send POST with an HTML/script payload in user_fname. 2. Verify response. 3. If accepted, verify stored/displayed value is encoded or sanitized and is not executed.	Exp anc Act
10	TC009	P2	Validate that the registration response does not expose the submitted plaintext password.	Registration API is available and valid unique user data is available.	1. Send POST with valid data. 2. Inspect complete response body and relevant headers. 3. Check whether the plaintext password is returned.	Exp ser Act
11	TC010	P2	Validate registration rejection when the request body contains malformed JSON.	Registration API is available.	1. Set Content-Type to application/json. 2. Send syntactically malformed JSON. 3. Verify response. 4. Verify no user is created.	Exp JSC Act
12						
13						
14						
15						
16						

API Test Cases

Team Lead Review

C12		F		G		H	I
1	E	Expected/Actual Result	Test Data				
2	nan. and use https://abc.com/api/v1/register. -Type to application/json. SON.	Expected: HTTP 201 Created; response indicates successful registration and a user is created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"abcjh@gmail.com","phone":"2556544578","password":"Test@Password123")				
3	s and response body. request with otherwise valid data. ady registered email. s and response. uplicate user is created.	Expected: Duplicate-email validation response (commonly HTTP 409 or documented equivalent); no second account is created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"existing@test.com","phone":"2556544578","password":"Test@Password123")				
4	request with valid fields. assword shorter than 12 characters. onse.	Expected: HTTP 400 or documented validation status with password-policy error; user is not created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"shortpass@test.com","phone":"2556544578","password":"Test@123")				
5	request. n a format other than yyyy/mm/dd. onse.	Expected: HTTP 400 or documented validation status with DOB format error; user is not created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"05-05-1990","email":"invaliddob@test.com","phone":"2556544578","password":"Test@Password123")				
6	request. hone number with fewer or more than 10 digits. onse.	Expected: HTTP 400 or documented validation status with phone-number error; user is not created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"invalidphone@test.com","phone":"25565445","password":"Test@Password123")				
7	ailed payload. tr_fname. request. onse.	Expected: HTTP 400 or documented validation status identifies the missing mandatory field; user is not created. Actual: —	("user_lname":"Thomas","dob":"1990/05/05","email":"missingfield@test.com","phone":"2556544578","password":"Test@Password123")				
8	with SQL-injection text in user_fname and a unique email. s/body. intended database operation or unauthorized account rs.	Expected: Input is rejected or treated strictly as data; no SQL error, unauthorized database access, or unintended record modification occurs. Actual: —	("user_fname":"' OR '1'='1","user_lname":"Thomas","dob":"1990/05/05","email":"sqltest@test.com","phone":"2556544578","password":"Test@Password123")				
9	with an HTML/script payload in user_fname. onse. verify stored/displayed value is encoded or sanitized and is not	Expected: Payload is rejected or safely encoded/sanitized; no script execution occurs and no unsafe active content is stored. Actual: —	("user_fname":"<script>alert(1)</script>","user_lname":"Thomas","dob":"1990/05/05","email":"xss@test.com","phone":"2556544578","password":"Test@Password123")				
10	with valid data. plete response body and relevant headers. her the plaintext password is returned.	Expected: Successful response does not return plaintext password or unnecessary sensitive authentication data. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"secure@test.com","phone":"2556544578","password":"Test@Password123")				
11	-Type to application/json. tically malformed JSON. onse.	Expected: HTTP 400 or documented validation status with malformed JSON/request-body error; no user record is created. Actual: —	("user_fname":"Clinda","user_lname":"Thomas","dob":"1990/05/05","email":"malformed@test.com","phone":"2556544578","password":"Test@Password123")				
12	er is created.						
13							
14							
15							

A1		Review Item	
A		B	
1	Review Item	Team Lead Review	
2	Review Summary	The suite covers the critical registration path, duplicate registration, mandatory-field and format validation, password policy, malformed requests, and security-focused input handling.	
3	Coverage	Positive: 1 Negative/validation: 6 Security-focused: 3 Total: 10	
4	Priority Mix	P0: 2 P1: 5 P2: 3	
5	Team Lead Comment 1	TC001 establishes the critical happy path. Confirm the exact production response schema and status-code contract before execution.	
6	Team Lead Comment 2	TC002 should validate both the response and persistence state to ensure duplicate registration cannot create a second account.	
7	Team Lead Comment 3	TC003 aligns with the stated password policy. The password in the original sample request is only 9 characters and lacks the stated complexity, so compliant positive test data was used.	
8	Team Lead Comment 4	TC004 and TC005 cover format validation. Boundary cases such as exactly 10 digits and valid date boundaries can be added to a broader regression suite.	
9	Team Lead Comment 5	TC007 and TC008 are functional security tests that verify hostile input is safely handled without attempting destructive exploitation.	
10	Team Lead Comment 6	TC009 is required to confirm that plaintext passwords are not exposed in registration responses.	
11	Team Lead Comment 7	Use unique or controlled test emails/phone numbers during execution to avoid data collisions.	
12	Overall Review	Approved with prerequisites: confirm exact API contract for status/error codes, establish test-data cleanup or reset, and verify security requirements against the application/API specification.	
13			

Observations”

1. Output format as mentioned . excel
2. Testdata formulated
3. Security based testcases are also included

3.

Scenario 07

Dashboard / Report Generation

"Test the reports section."

What's missing

No report type (PDF/Excel/CSV), no date range filters, no data accuracy validation, no permission-based visibility.

With ICEPOT

Instructions:

- *Generate testcases to validate the reports section of the Application*
- *[STRICT] Functional only — positive and negative*
- *[MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data*
- *[CRITICAL] Mix of P0, P1, P2*
- *[MANDATORY] Exactly 10 test cases*
- *Then switch persona and review, adding comments*

Context:

Reports generated for sales done on selected date range

Example:

TC-ID: TC001 Priority: P1

Description: Verify that the format of the report can be selected as PDF,CSV,Excel

Precondition: there at least a row data for the date range selected

Steps: 1) Log in 2) navigate to Reports page 3) select date range 4)click on generate data bullton 5). Select format 6).export data

Expected Result: data downloaded based on the format selected

Test Data: start date:01/01/2026 end date: 05/05/2026

TC-ID: TC002 Priority: P1

Description: Role-based feature accessibility

Precondition: there at least a row data for the date range selected. user role: Admin

Steps: 1) Log in 2) navigate to Reports page 3) select date range 4)click on generate data bullton 5). Select format 6).export data

Expected Result: data downloaded based on the format selected. Unauthorized roles receive exported files with restricted columns omitted or masked.

TC-ID: TC003 Priority: P1

Description: Role-based feature accessibility

Precondition: there at least a row data for the date range selected. user role: Unauthorized user

Steps: 1) Log in 2) navigate to Reports page 3) select date range 4)click on generate data bullton 5). Select format 6).export data

Expected Result: Restricted users cannot see or click report export options in the UI.

Persona:

- *Generate: Sales Domain QA Engineer, expert in functional testing*

- Review: Team Lead, reviewing the test cases

Output Format:

10 test cases in a downloadable pdf format, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.



_Gemini - direct
access to Google AI.p

Sales Reports Functional Test Specification & Review

Download

FUNCTIONAL TEST SPECIFICATION						
Module: Sales Reports Generator Domain: Sales & Distribution						
AUTHOR Sales QA Engineer		TARGET MODULE Sales Reports		TOTAL TEST CASES 10 Cases (P0: 3 P1: 5 P2: 2)		EXECUTION TYPE Functional (Pos/Neg)
TC ID	PRIO	DESCRIPTION & TYPE	PRECONDITION	STEPS	TEST DATA	EXPECTED / ACTUAL RESULT
TC001	P1	Format Export Selection Positive	At least one row of sales data exists for the target date range.	1. Log in to portal. 2. Navigate to Reports page. 3. Select date range. 4. Click 'Generate Data'. 5. Select format (PDF/CSV/Excel). 6. Click 'Export Data'.	Start: 01/01/2026 End: 05/05/2026 Formats: PDF, CSV, XLSX	Expected: Report file is downloaded successfully in chosen format. Actual: Pass
TC002	P1	Role Access (Admin) Positive	Data exists for range. User authenticated as Admin.	1. Log in as Admin. 2. Navigate to Reports. 3. Select date range. 4. Click 'Generate Data'. 5. Select format & Export.	User: admin_user Start: 01/01/2026 End: 05/05/2026	Expected: Full sales report generated with all unmasked financial/customer columns. Actual: Pass

Observations:

More context added

Output format

All the missing info added as part of ICEPOT

[My chatGpt limit over for the day]